Lab04 - Neural Network in PyTorch

Objectives:

1. Learn how to construct a standard neural network in PyTorch
2. Learn how to handle image data

Task:

We will perform the following steps in order:

1. Load and transform the cifarset dataset using torchvision
2. Define a Convolutional Neural Network using torch
3. Define a loss function
4. Train the network
5. Test the network

Content:

1. The CIFAR10 dataset
   A. TorchVision: Dataset
   B. TorchVision: Transformations
   C. TorchVision: DataLoader
2. Building Regular Neural Network
   A. The torch.nn.Module library
   B. Functions in torch.nn.functional and torch
3. Training the model
4. Predicting Using the Trained Model

In [ ]:
```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).
```

In [ ]:
```
cd "gdrive/My Drive/UCCD3074_Labs/UCCD3074_Lab4"
```

```
[Errno 2] No such file or directory: 'gdrive/My Drive/UCCD3074_Labs/UCCD3074_Lab4'
/content/gdrive/My Drive/UCCD3074_Labs/UCCD3074_Lab4
```

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

from torch.utils.data import DataLoader

from torchsummary import summary
```

```
from cifar10 import CIFAR10

import torchvision
import torchvision.transforms as transforms

%load_ext autoreload
%autoreload 2
```

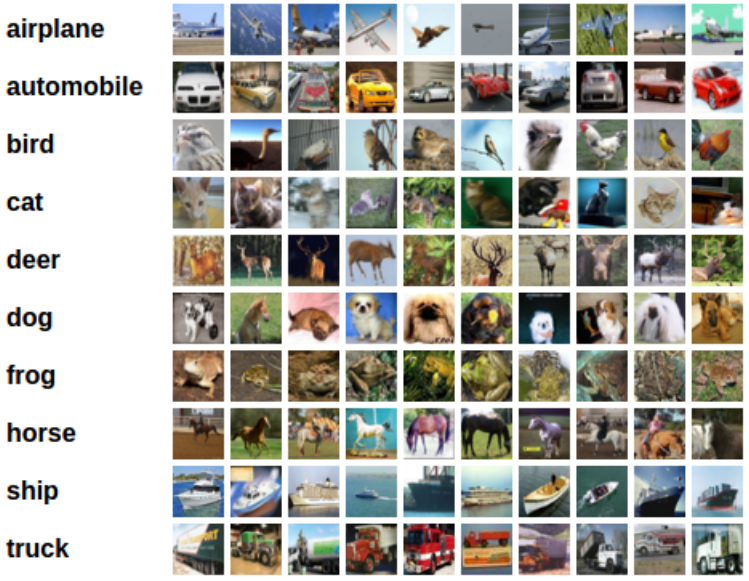
```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

---

1. The CIFAR10 dataset

For this tutorial, we will use the CIFAR10 dataset. The CIFAR-10 dataset consists of 60000 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images. However for speed consideration, we shall subsample 10000 training images and 2000 training images.

The following figure shows 10 randomly selected samples for each class. This is a **multi-class classification** where there is only one object per image. The images in CIFAR-10 are of size (3, 32, 32), i.e. 3-channel color images of 32x32 pixels in resolution.



`torchvision.datasets`

First, we load the CIFAR10 dataset, one of the many computer vision datasets listed in `torchvision.datasets` . The dataset is an object of type `torch.utils.data.Dataset` . To reduce the training time, we have create a wrapper function `CIFAR10` in the module *cifar10.py* that loads only a subset of the CIFAR10 library.

In [ ]:
```
trainset = CIFAR10(root=".", train=True, num_samples=10000, download=True)
testset  = CIFAR10(root=".", train=False, num_samples=2000, download=True)
```

```
Files already downloaded and verified
Files already downloaded and verified
```

The classes can be found in the `classes` attribute of the dataset object

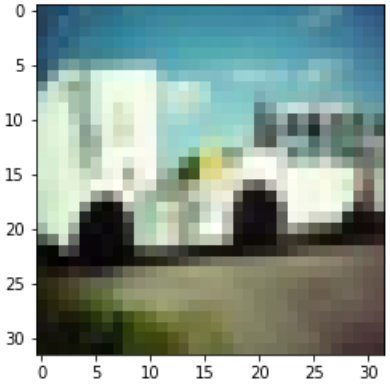
In [ ]:
```
classes = trainset.classes
print('classes =', classes)
print('Number of training set:', len(trainset))
print('Number of test samples:', len(testset))
```

```
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
Number of training set: 10000
Number of test samples: 2000
```

We can access sample *i* directly by indexing the dataset `trainset[i]` which returns the tuple `(image, label)` .

In [ ]:
```
img, label = trainset[0]
plt.imshow(img)
print('label =', label, ' class =', classes[label])
```

```
label = 9  class = truck
```



Each sample in the dataset is a numpy array of shape `(height, width, channel)` . The pixel values ranges from 0 to 255. In addition, the data type of the image is `uint8` .

In [ ]:
```
print('\nType of img:', type(img))
print('Type of items in img:', img.dtype)

print('Shape of img:', img.shape)
print('Range of img:', img.min(), 'to', img.max())
```

```
Type of img: <class 'numpy.ndarray'>
Type of items in img: uint8
Shape of img: (32, 32, 3)
Range of img: 0 to 255
```

---

2. TRANSFORMING THE INPUT IMAGE

Composing the Transformation Pipeline

The torchvision package contains the `torchvision.transforms` library which contains the set of common image transformations. You can create a pipeline (series) of transformations by chaining them together using the command `torchvision.transform.Compose` .

For example, the following code creates the transformation pipeline:

1. Convert the input image (numpy array of shape (H, W, C)) into a PIL image (`transform.ToPILImage()`).
   Many of various image transformation functions in `torchvision.transform` can only work on PIL, but not numpy arrays.
2. Randomly flip the image horizontally (`transform.RandomHorizontalFlip()`)
3. Randomly sample a sub-image of shape (24, 24) from the image (`transform.RandomCrop()`)

In [ ]:
```
transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop((24, 24))
])
```

Given an image, we can subject it to `transform` that will then perform the series of specified transformations.

In [ ]:
```
img, label = trainset[1]
plt.imshow(img)
plt.title('Original image', fontsize=16)
_ = plt.axis("off")
```

Original image

In [ ]:
```
transformed_img = transform(img)
plt.imshow(transformed_img)
plt.title('Transformed image')
_ = plt.axis("off")
```

Transformed image

Transforming the CIFAR dataset

For the CIFAR10 dataset, we shall perform the following transformation:

1. Convert the type from numpy *array* to a PyTorch *tensor* and change the dimension from `(H, W, C)` to `(C, H, W)` (`torchvision.transforms.ToTensor`). This is a mandatory step.
2. Normalize the data with mean and standard deviation for each channel: `output[c] = (input[c] - mean[c])/std[c]` (`torchvision.transforms.Normalize`)

In [ ]:
```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (1.0, 1.0, 1.0))
])
```

Before the transformation, the image has a shape of `(h, w, c)` and the pixel ranges between 0 and 255.

In [ ]:
```
img, label = trainset[1]
print('Original image shape:', img.shape)
print(f'Range of pixel values: {img.min():d}-{img.max():d}')
```

```
Original image shape: (32, 32, 3)
Range of pixel values: 6-255
```

After the transformation. the image has a shape of `(c, h, w)` and the pixel values are normalized with a mean and standard deviation of roughly 0.5 and 1.0, respectively.

In [ ]:
```
x = transform(img)
print('Original image shape:', x.shape)
print(f'Range of pixel values: {x.min():.2f}-{x.max():.2f}')
```

```
Original image shape: torch.Size([3, 32, 32])
Range of pixel values: -0.48-0.50
```

Reloading the dataset with transformation

Now that we have defined the transformation pipeline, we shall reload our dataset with `transform=transform` to apply the transformation upon loading.

In [ ]:
```
trainset = CIFAR10(train=True,  transform=transform, num_samples=10000, download=True)
testset  = CIFAR10(train=False,  transform=transform, num_samples=2000, download=True)
```

```
print('\nNumber of training samples:', len(trainset))
print('Number of test samples:', len(testset))
```

```
Files already downloaded and verified
Files already downloaded and verified

Number of training samples: 10000
Number of test samples: 2000
```

Note that the the loaded image now has a shape of (C x H x W), the range is between [-1, 1], the type is *torch.FloatTensor*.

In [ ]:
```
img, label = trainset[0]
print('Shape of img:', img.shape)
print(f'Range of img: {img.min().item():.2f} to {img.max().item():.2f}')
print('Type of img:', type(img))
print('Type of items in img:', img.dtype)
```

```
Shape of img: torch.Size([3, 32, 32])
Range of img: -0.48 to 0.50
Type of img: <class 'torch.Tensor'>
Type of items in img: torch.float32
```

---

3. Batch Loading with BatchLoader

Minibatch Gradient Descent (MGD) loads the data in batches. To do that, we need to do the following:

1. Batch the data for parallel processing
2. Shuffle the data

`DataLoader` `torch.utils.data.DataLoader` is an iterator that provides the above two features. The following code uses a dataloader to load a batch size of 4 (`batch_size=4`). The datasets are shuffled at the beginning of each epoch (`shuffle=True`). A total of 4 workers are used to load the data in parallel (`num_workers=4`).

In [ ]:
```
trainloader = DataLoader(trainset, batch_size=4, shuffle=True, num_workers=2)
testloader  = DataLoader(testset, batch_size=4, shuffle=True, num_workers=2)
```

The following loads one batch data randomly from the training set. Since `trainloader` is an iterator, you need to use `iter` to retrieve a batch sample.

In [ ]:
```
dataiter = iter(trainloader)
imgs, labels = dataiter.next()

print('Shape of imgs:', imgs.shape)
print('Shape of labels:', labels.shape)
```

```
Shape of imgs: torch.Size([4, 3, 32, 32])
Shape of labels: torch.Size([4])
```

---

4. Building Regular Neural Network

In this section, we shall build the following 2-layered neural network:

Layer	Name	Operation	Input Shape	OutputShape
-		view	(?, 3, 32, 32)	(?, 3072)
1	`fc1`	Linear, 50 neurons	(?, 3072)	(?, 50)
	`relu1`	Relu	(?, 50)	(?, 50)
2	`fc2`	Linear, 10 neurons, no activation	(?, 50)	(?, 10)

- `fc1` is the hidden layer. It contains 50 neurons. It uses relu for its activation
- `fc2` is the output layer. It contains 10 neurons which is equivalent to the number of classes in the task. We will not use any activation for the second layer for reasons that will be clear later.

Coding the network

The neural network architectures in PyTorch can be defined as a sub-class of `nn.Module`. We shall do

1. define all the layers inside the constructor `__init__(self)`. The modules for the layers are defined in torch.nn which include the convolutional layers, pooling layers, linear (fully connected) layers, etc.
2. perform forward propagation steps in the function `forward(self, x)`.

The following shows how to define a CNN called `Net`.

In [ ]:
```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3*32*32, 50)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(50, 10)

    def forward(self, x):
        # flatten each image
        x = x.view(x.size(0), -1)

        # forward propagation for layer 1
        x = self.fc1(x)
        x = self.relu1(x)

        # forward propagation for layer 2
        x = self.fc2(x)

        return x
```

In [ ]:
```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3*32*32, 50)
        self.fc2 = nn.Linear(50, 10)

    def forward(self, x):
        # flatten each image
        x = x.view(x.size(0), -1)
```

```
        # forward propagation for layer 1
        x = self.fc1(x)
        x = F.relu(x)

        # forward propagation for layer 2
        x = self.fc2(x)

        return x
```

- line 1, 8, 2: The class of the network inherits from `nn.Module` and overloads the the function `__init__` and `forward` .
- line 2-6: Initialize the network
  - This function **creates the layer objects** in our network upon initialization. **No computational graph is constructed** yet at this point of time.
  - line 4: Creates the 1st linear layer, `self.fc1` .
  - line 5: Creates the activation function for layer 1, `self.relu1` .
  - line 6: Creates the 2nd linear layer, `self.fc2` .
  - Note that `self.fc1` , `self.relu1` and `self.fc2` are all `nn.Modules` objects.
- line 8-17:
  - Perform forward propagation and **creates the computational graph**
  - line 10: flattens all batch images. Here, we maintain the first dimension (`x.size(0)`) and flattens the 2nd dimension onwards. As a result, the tensor's shape changes from (B, 3, 32, 32) to (B, 3072)
  - line 13-14: performs forward propagation through layer 1
  - line 17: performs forward propagation through layer 2

Displaying the network

When you print the network, it will only list out the layers following the sequence declared in `__init__` . The does **not** reflect the actual structure of the computational graph built when you invoke the `__forward__` function.

In [ ]:
```
net = Net()
print(net)
```

```
Net(
  (fc1): Linear(in_features=3072, out_features=50, bias=True)
  (relu1): ReLU()
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)
```

Exercise: Try to swap `fc1` and `relu`` declaration in the `__init__` function of Net` above. Then, print the network. Observe that *printing* the the network does not reflect the true layout of the network.

In [ ]:
```
print(net)
```

To observe the correct layout of the network, you can use **`torchsummary.summary`** which also outputs the shape of the activation map given input of a specified size.

To use `summary` , you need to provide the network object, the input size (shape of one input sample), the batch size (optional) and device ("cuda" or "cpu", optional).

In [ ]:
```
net = Net()
summary(net, input_size=(3, 32, 32), batch_size=4, device="cpu")
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Linear-1                    [4, 50]         153,650
              ReLU-2                    [4, 50]               0
            Linear-3                    [4, 10]             510
================================================================
Total params: 154,160
Trainable params: 154,160
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.05
Forward/backward pass size (MB): 0.00
Params size (MB): 0.59
Estimated Total Size (MB): 0.64
----------------------------------------------------------------
```

Testing the network on the fly

In the process of constructing the network, we can apply some random input to the network even when it is still untrained. This allows us to test your network coding as we define them.

Exercise: Create a random tensor with 4 samples and check if the dimension of the output is correct. What should be the shape of the output tensor?

In [ ]:
```
net = Net()

inp = torch.rand(4, 3, 32, 32)
out = net(inp)

print('Shape of out:', out.shape)
print('Out:\n', out)
```

```
Shape of out: torch.Size([4, 10])
Out:
 tensor([[-0.0176,  0.1281, -0.2480, -0.1873, -0.1203, -0.1197,  0.2208, -0.1923,
         -0.2644, -0.2054],
        [ 0.1072,  0.2147, -0.2304, -0.0975, -0.2266, -0.1999,  0.0734, -0.2760,
         -0.1772, -0.1500],
        [ 0.1294,  0.2125, -0.2243, -0.1244, -0.1307, -0.2689,  0.1244, -0.2280,
         -0.2145, -0.1588],
        [ 0.0815,  0.2163, -0.3722, -0.1444, -0.1224, -0.1541,  0.1465, -0.1426,
         -0.1550, -0.0972]], grad_fn=<AddmmBackward>)
```

Module vs Functional

`nn.module` classes have attributes, e.g., `fc1` has internal attributes like `self.weight` and `self.bias` . However, some layers , e.g., *sigmoid*, *relu*, *dropout*, etc., are attributeless. Rather than explicitly create a layer for these attributeless layers, we can use a **functional** approach to define these layer. You can use the function either in **`torch.nn.functional`** (e.g., `nn.functional.relu` , `nn.functional.avg_pool2d` , `nn.functional.softmax`) or **`torch`** (e.g., `torch.sigmoid` , `torch.sin`) to perform forward propagation.

To use the functional modules, you just have to call the function directly just like any function without having to declare them first (that's why they are called functionals. For example:

```
import torch.nn.functional as F

...
class Net(nn.Module):
    def __init__():
        ...

    def forward():
        ...
        x = F.relu(x)
```

Note that all layers implemented using functional are invisible to `print` and `torchsummary.summary` . So, next time when you print the layers of this network, remember that it may be telling you the whole story.

Exercise.

1. Modify the class `Net` above to implement the `relu1` using the functional approach.
2. Use a `print` statement to display the network. Observe that it does not print the functional `relu` layer although it is in th network.
3. Use a `torchsummary.summary` statement to display the network. Observe that it does not print the functional `relu` layer although it is in th network.

In [ ]:
```
net = Net()
print(net)
```

```
Net(
  (fc1): Linear(in_features=3072, out_features=50, bias=True)
  (relu1): ReLU()
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)
```

In [ ]:
```
summary(net, input_size=(3, 32, 32), batch_size=4, device="cpu")
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Linear-1                    [4, 50]         153,650
              ReLU-2                    [4, 50]               0
            Linear-3                    [4, 10]             510
================================================================
Total params: 154,160
Trainable params: 154,160
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.05
Forward/backward pass size (MB): 0.00
Params size (MB): 0.59
Estimated Total Size (MB): 0.64
----------------------------------------------------------------
```

---

4. Training the model

Now, we are ready to train our model.

Create the network

Now, let's create the network and transfer them to the GPU if available. **Remember to enable the GPU for this notebook** by setting `Edit > Notebook settings > Hardware accelerator to GPU`.

In [ ]:
```
net = Net()
if torch.cuda.is_available():
    net = net.cuda()
```

Define the optimizer

To update the parameters, we can use the `torch.optim` package which contains various formulas for optimizing the parameter. For this practical, we use `nn.optim.SGD` optimizer ($w := w - \alpha \frac{dcost}{dw}$) to update `net.parameters`. The update (`optimizer.step()`) will commence only after forward-backward propagation.

In [ ]:
```
optimizer = optim.SGD(net.parameters(), lr=0.001)
```

Define the cost function

Since we are performing **multi-class classification**, we shall use the cross entropy loss. There are at least 3 ways to implement the loss function:

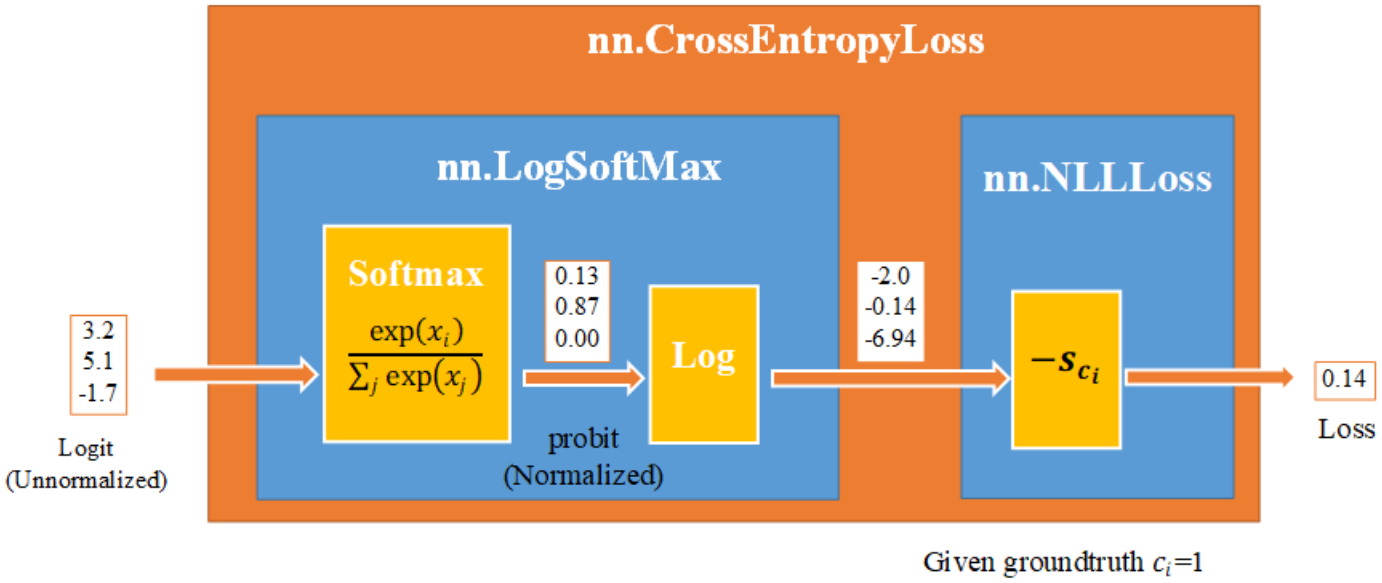
1. log softmax activation + negative log likelihood

    A. Use `log_softmax` activation for the last (classification) layer.
    B. Use `NLLLoss` (negative log likelihood) to compute the cost. Here, the input to the function is the *log likelihood* values. Hence, the input to `NLLLoss` must have applied `log` prior to calling the function.
2. softmax activation + negative likelihood:

    A. Use `softmax` function for the last (classification) layer
    B. Perform `log` on the output of the layer
    C. Use `NLLLoss` (negative log likelihood) to compute the cost.

1. No activation + cross entropy loss
    A. No activation for the last (classification) layer
    B. Use `CrossEntropyLoss` which will implements both `log_softmax + NLLLoss` in a single function or module.

Since our last layer has no activation, we shall be implementing the 3rd method. We shall use the `nn.CrossEntropyLoss` as criterion to compute the loss function. The block operations of `nn.CrossEntropyLoss` is sumarized in the diagram below:



In [ ]:
```
criterion = nn.CrossEntropyLoss()
```

Set to training mode

The following code tells the model that you are in training mode. Some layers like dropout, batchnorm etc. behave differently during training and testing.

In [ ]:
```
net.train()
```

Out[ ]:
```
Net(
  (fc1): Linear(in_features=3072, out_features=50, bias=True)
  (relu1): ReLU()
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)
```

Train the model

Now we are ready to train our model for 6 epochs.

In [ ]:
```
num_epochs = 6
loop_per_val = 500

for e in range(num_epochs):

    running_loss = 0
    running_count = 0

    for i, (inputs, labels) in enumerate(trainloader):

        # transfer input to GPU
        if torch.cuda.is_available():
            inputs = inputs.cuda()
```

```
            labels = labels.cuda()

        # set grad to zero
        net.zero_grad()

        # forward propagation
        outputs = net(inputs)

        # compute loss
        loss = criterion(outputs, labels)

        # backward propagation
        loss.backward()

        # update model
        optimizer.step()

        # compute running loss and validation
        running_loss += loss.item()
        running_count +=1

        # display the averaged loss value
        if i % loop_per_val == loop_per_val - 1:
            train_loss = running_loss / loop_per_val
            running_loss = 0.
            running_count = 0
            print(f'[Epoch {e+1:2d}/{num_epochs:d} Iter {i+1:5d}/{len(trainloader)}]: train_loss = {train_loss:.4f}')
```

```
[Epoch  1/6 Iter   500/2500]: train_loss = 2.2820
[Epoch  1/6 Iter  1000/2500]: train_loss = 2.2301
[Epoch  1/6 Iter  1500/2500]: train_loss = 2.1875
[Epoch  1/6 Iter  2000/2500]: train_loss = 2.1409
[Epoch  1/6 Iter  2500/2500]: train_loss = 2.1179
[Epoch  2/6 Iter   500/2500]: train_loss = 2.0868
[Epoch  2/6 Iter  1000/2500]: train_loss = 2.0548
[Epoch  2/6 Iter  1500/2500]: train_loss = 2.0351
[Epoch  2/6 Iter  2000/2500]: train_loss = 1.9816
[Epoch  2/6 Iter  2500/2500]: train_loss = 1.9694
[Epoch  3/6 Iter   500/2500]: train_loss = 1.9613
[Epoch  3/6 Iter  1000/2500]: train_loss = 1.9464
[Epoch  3/6 Iter  1500/2500]: train_loss = 1.9241
[Epoch  3/6 Iter  2000/2500]: train_loss = 1.8878
[Epoch  3/6 Iter  2500/2500]: train_loss = 1.8982
[Epoch  4/6 Iter   500/2500]: train_loss = 1.8604
[Epoch  4/6 Iter  1000/2500]: train_loss = 1.8542
[Epoch  4/6 Iter  1500/2500]: train_loss = 1.8583
[Epoch  4/6 Iter  2000/2500]: train_loss = 1.8453
[Epoch  4/6 Iter  2500/2500]: train_loss = 1.8578
[Epoch  5/6 Iter   500/2500]: train_loss = 1.8255
[Epoch  5/6 Iter  1000/2500]: train_loss = 1.8148
[Epoch  5/6 Iter  1500/2500]: train_loss = 1.8235
[Epoch  5/6 Iter  2000/2500]: train_loss = 1.7957
[Epoch  5/6 Iter  2500/2500]: train_loss = 1.7654
[Epoch  6/6 Iter   500/2500]: train_loss = 1.7702
[Epoch  6/6 Iter  1000/2500]: train_loss = 1.7645
[Epoch  6/6 Iter  1500/2500]: train_loss = 1.7637
[Epoch  6/6 Iter  2000/2500]: train_loss = 1.7529
[Epoch  6/6 Iter  2500/2500]: train_loss = 1.7814
```

- Line 2, 6-7, 31-40: Computes the averaged loss over 500 iterations
- Line 9: load the batch sample, namely `inputs` with a shape of (4, 3, 32, 32) and `labels` of size (4,). where batch_size = 4 and #channel = 3.
- Line 20: forward propagation to compute the scores for current model
- Line 23: compute the loss value
- Line 26: backpropagation. The gradients for `net.parameters` are computed.
- Line 29: updates `net.parameters` based on the gradient values computed from backpropagation

Retraining the model

Note that if you simply re-run the training cell directly above, it will continue the previous training the model. You can observe it by seeing that the starting cost value is very low and continues from the previous training session. To train the model from scratch again, you can create a new network by running the code `net = Net()` to start clearn.

---

5. Predicting Using the Trained Model

We can use our trained model to predict on the testset. Expected accuracy is around 38%. The performance is better than random guess. If we randomly guess, we can expect the performance is around 10% since there are 10 classes and the classes are well distributed.

Set to evaluation mode

The following code tells the model that you are in evaluation or testing mode. Some layers like dropout, batchnorm etc. behave differently during training and testing.

In [ ]:
```
net.eval()
```

Out[ ]:
```
Net(
  (fc1): Linear(in_features=3072, out_features=50, bias=True)
  (relu1): ReLU()
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)
```

Evaluate the model

Now we are ready to evaluate our model.

In [ ]:
```
# running_correct to accumulate number of correct predictions
running_corrects = 0

for inputs, labels in testloader:

    # transfer to GPU
    if torch.cuda.is_available():
        inputs = inputs.cuda()
        labels = labels.cuda()

    # do not want to compute the gradient
    with torch.no_grad():

        # forward propagation to get class scores
        scores = net(inputs)

        # get the predicted class
```

```
        _, preds = torch.max(scores, 1)

        # compute the running correct
        running_corrects += (preds == labels).double().sum()

accuracy = running_corrects / len(testloader.dataset)*100
print(f'Accuracy = {accuracy:.2f}%')
```

```
Accuracy = 37.70%
```

- Line 4: Loads the `input` (shape = (4, 3, 32, 32)) and `target` (shape=(4,))
- Line 12: Since we are no longer training the model, there is no need to the compute gradient anymore. The wrapper `with torch.no_grad()` temporarily sets all `requires_grad` flag to false. This will save more memory and speed up the process.
- Line 15: Perform foward propagation to get the class scores. `scores` is a FloatTensor of size (4, 10). Batch size is 4 and there are 10 classes.
- Line 18: Gets the predicted class for each score. `preds` is a LongTensor of size (4,) indicating the class index for each batch sample.
- Line 2 and 21 is used to accumulates the number of correctly predicted samples throughout the batch iteration.
- Line 23: Computes the testing accuracy

Analysis

The standard NN should achieve a test accuracy of around 37%. So, is it learning anything useful? The answer is yes, because 37% is still better than a random guess where you can expect a performance of around 10% because there are 10 classes in the network. Admittedly, the performance is far from satisfactory and this not the best that deep learning can do. In the next lab, we shall improve the performance of CIFAR10 using CNN networks.

Conclusion

In this lab, you have implemented neural network using PyTorch. In the process, you have learnt how to

1. Use the **data loader** to load batch data
2. Implement a **transformation pipeline** to preprocess images using `torchvision.transforms` .
3. Use a PyTorch's **optimizer** (`nn.optim.SGD`) to train the network
4. Define a **custom neural network** class using `nn.Module` .
5. How to implement the **cost function**
6. Understand the difference between `nn.Module` and `nn.functional` .